

Docker & Kubernetes

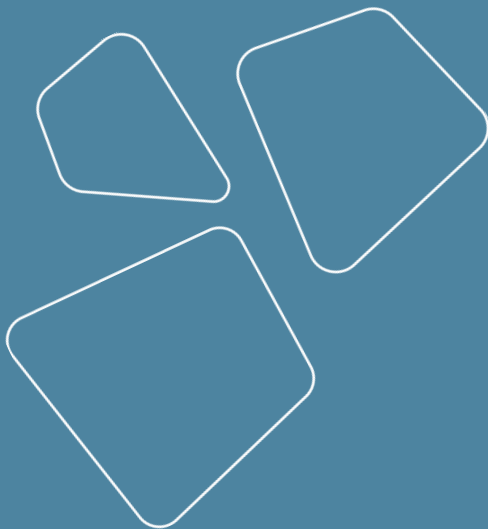
Vladislav Goncharenko
Demid Gaibov



MIPT, spring 2026

Outline

1. Virtualization
2. Docker
 - a. Podman
 - b. Compose
3. Kubernetes



Virtualization

girafe
ai

00

Какую проблему решаем?

Нужно поддерживать несколько приложений на одной машине, у которых

- Разные версии python-библиотек
- Разные версии системных библиотек
- Разные дистрибутивы OS

Ответы:

- VM
- Docker
- Podman



Runtime environments

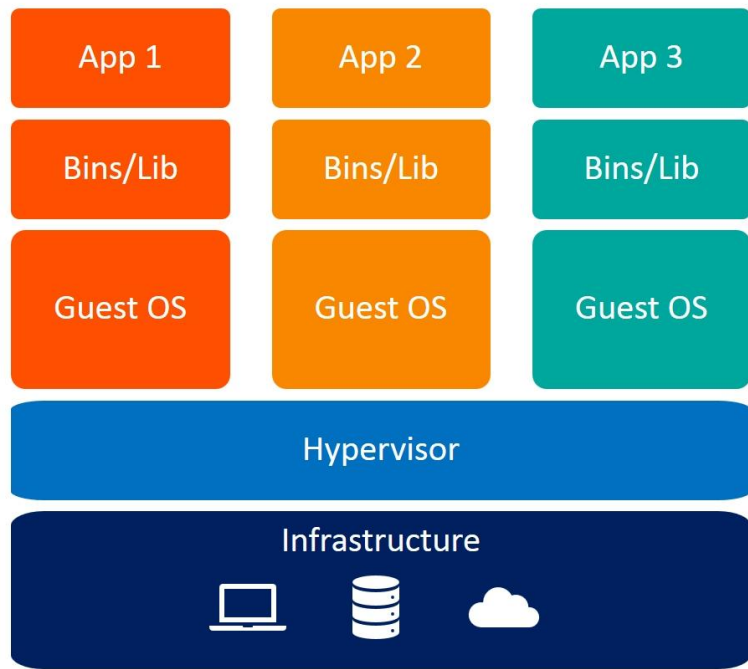
- Bare metal
 - runs on host OS
- Virtual machines
 - Hardware virtualization
 - use same hardware directly
 - Software virtualization
 - emulate hardware for guest OS
- Containers
 - emulate system calls with host OS code

Hypervisors

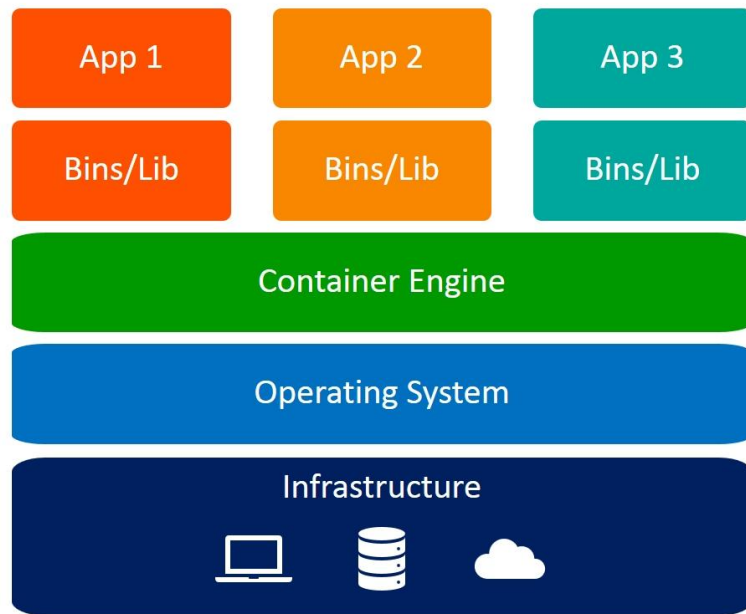
- VirtualBox by Oracle
- QEMU
- UTM
- VMware
- Parallels



Virtual machine vs Container



Virtual Machines



Containers



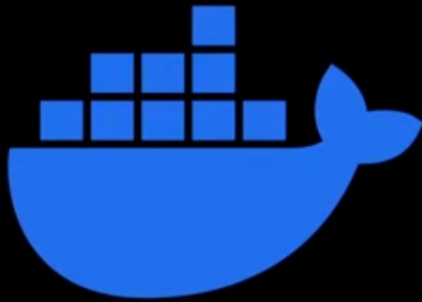
Docker

girafe
ai

01

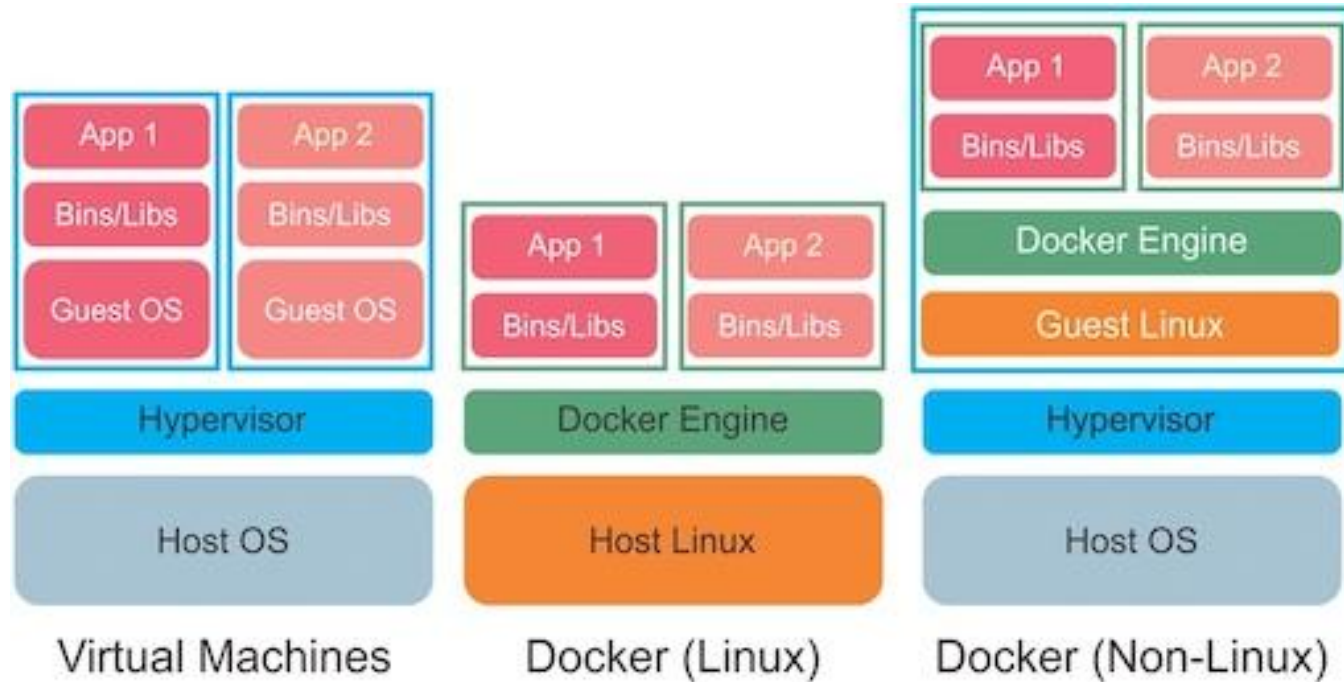


WHALE WHALE WHALE...



WHO DO WE HAVE HERE?

Docker uses Linux

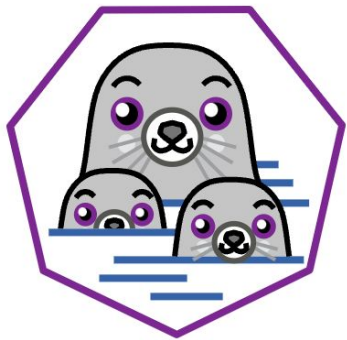


[Article on topic](#)

FOSS alternative

<https://podman.io/>

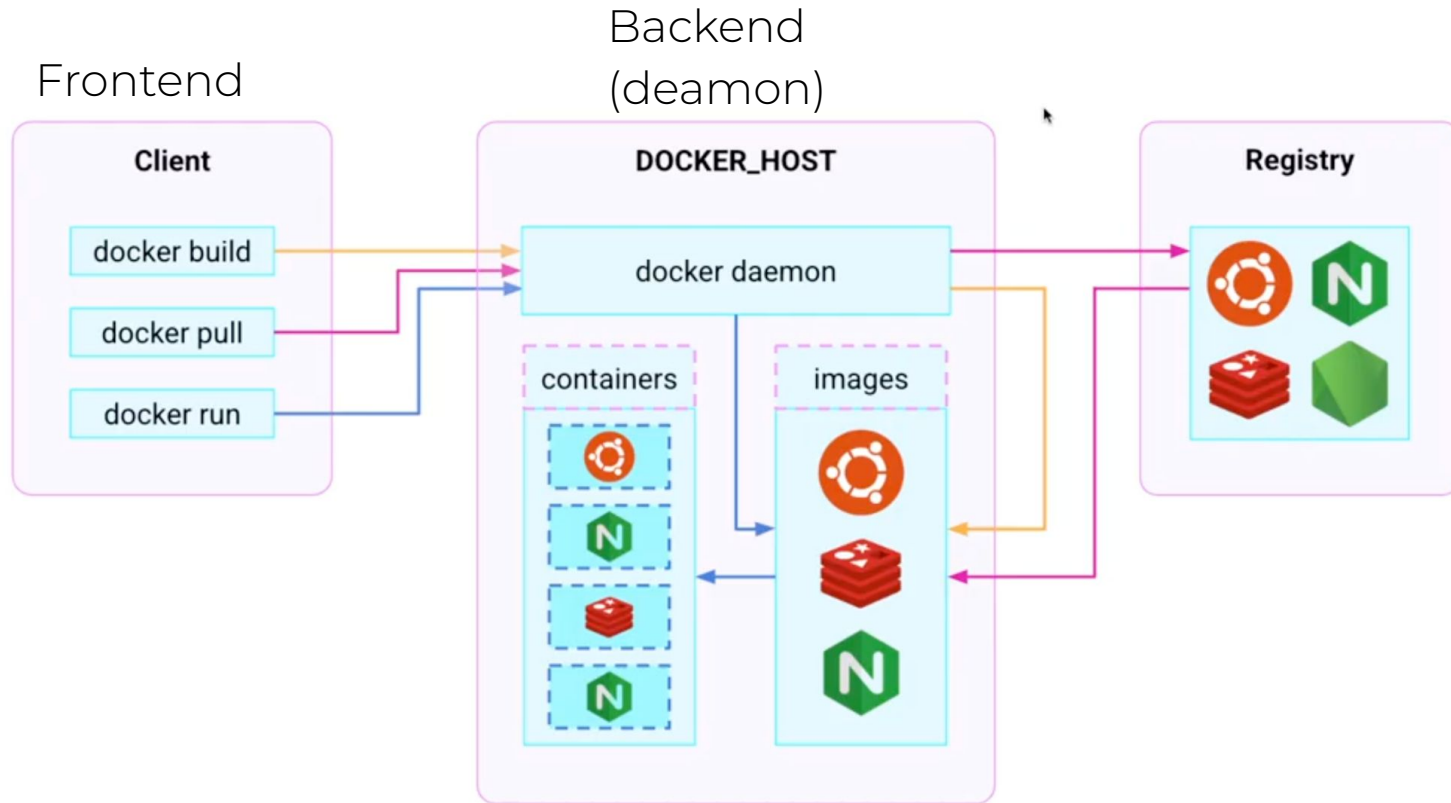
Written on Go as well by RedHat



podman



Docker structure



Container lifecycle

- Dockerfile
 - instruction how to make an image
- Image
 - snapshot of the system on some stage
 - uses UnionFS
- Container
 - process on host OS having particular state



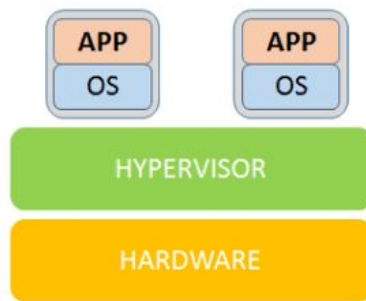
VM

Программа-гипервизор
эмулирует

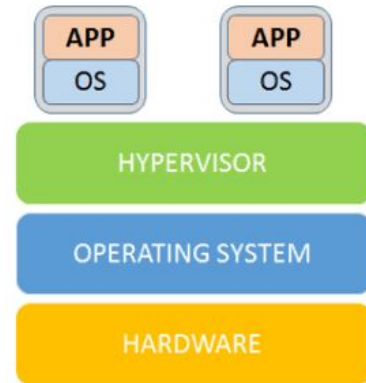
- Гостевую OS
- Приложение в ней

Примеры:

- Type 1: [Hyper-V](#)
- Type 2: [VirtualBox](#)



Type 1 Hypervisor

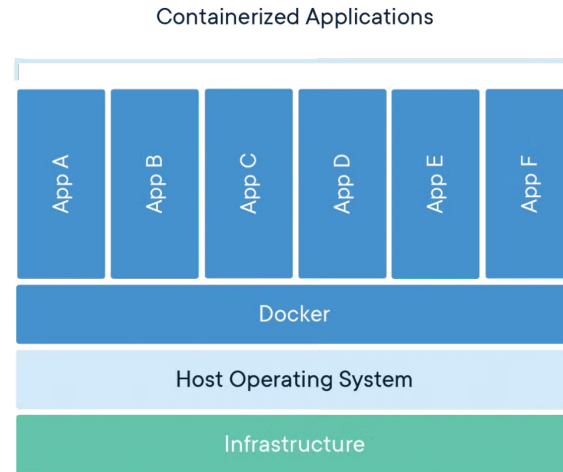
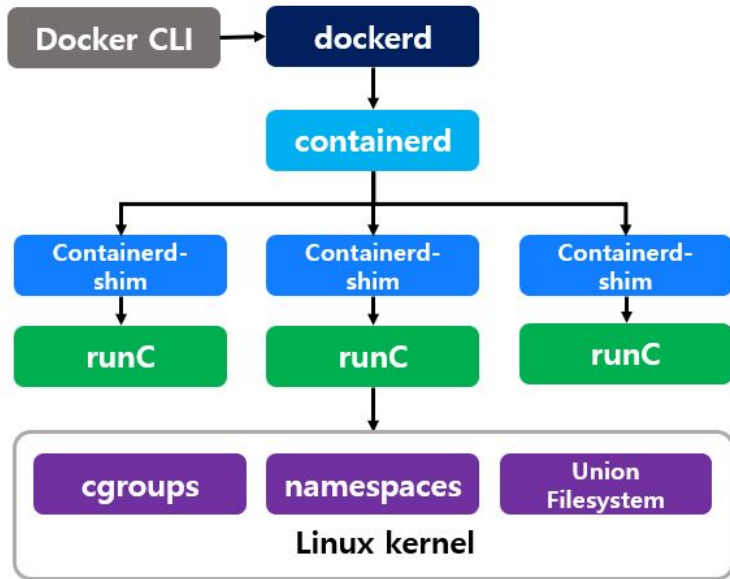


Type 2 Hypervisor



Docker

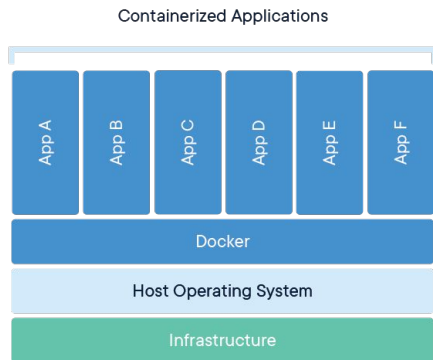
Запуск и изоляция контейнера выполняются силами Linux, ядро остается общим



Минусы VM и Docker

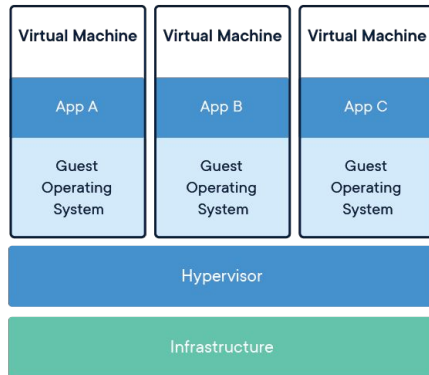
Docker:

- Только Linux ядро
- Есть проблемы безопасности
- Сложность в регулировании общих ресурсов (сеть, ...)

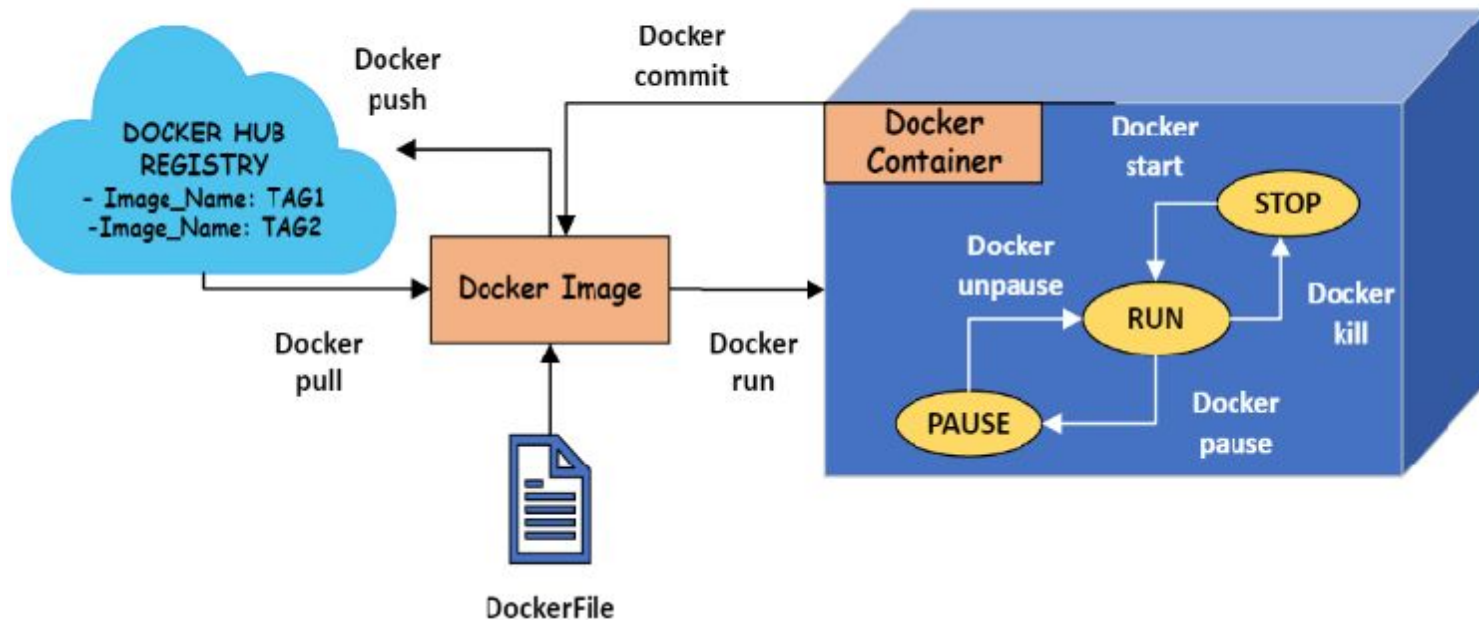


VM:

- Эмуляция требует ресурсов
- Тяжело взаимодействовать между VM
- Проблемы с подключением устройств



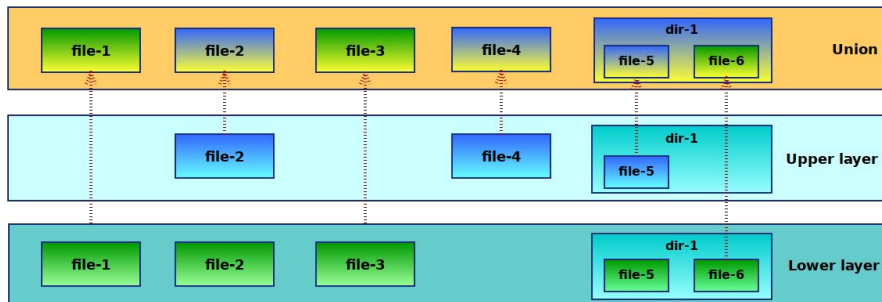
Процесс работы с Docker



Docker Image



Unioning filesystem



Dockerfile

- FROM - базовый образ
- RUN - выполнить команду
- ENTRYPOINT, CMD - команда для выполнения при старте
- ENV - указать переменную окружения
- ADD, COPY - добавить файлы
- WORKDIR - сменить директорию
- ARG - прокидывание аргументов во время сборки

[Reference](#)

Docker: Keywords

- [Volume](#) - как добавить дисковое пространство
- [Network](#) - настройка сети
- [Multi-Stage Build](#) - оптимизация размера образа
- [Secret](#) - использование секретов во время сборки
- [Build Context](#) - какие файлы видны при сборке, как это настроить
- [Podman](#), [colima](#) - open-source аналог



Docker Compose

Хотим запускать несколько контейнеров, которые:

- управляются по кнопке
- могут общаться по сети
- имеют зависимости по запуску

```
1  services:
2    recommendation-engine:
3      image: ubuntu
4      tty: true
5      volumes:
6      - DataVolume:/DataVolume
7      labels:
8        brownout.feature: "optional"
9      deploy:
10        replicas: 2
11        restart_policy:
12          condition: none
13        placement:
14          constraints: [node.role == worker]
15
16    user-db:
17      image: weaveworksdemos/user-db
18      hostname: user-db
19      deploy:
20        placement:
21          constraints: [node.role == manager]
```



Kubernetes

girafe
ai

02

Зачем что-то еще?

Хотим:

- Масштабироваться по нодам
- Поддерживать отказоустойчивость
- Простой Service Discovery
- Возможность постепенного деплоя (Rolling Update)

Решения:

- Docker Swarm
- Kubernetes



Service-level agreement (SLA)

There is a rule of

Availability %	Friendly Name	Downtime		
		per Year	per Month	per Week
90%	one nine	36.5 days	72 hours	16.8 hours
99%	two nines	3.65 days	7.2 hours	1.68 hours
99.5%	--	1.83 days	3.6 hours	50.4 minutes
99.9%	three nines	8.76 hours	43.8 minutes	10.1 minutes
99.95%	--	4.38 hours	21.56 minutes	5.04 minutes
99.99%	four nines	52.56 minutes	4.32 minutes	1.01 minutes
99.999%	five nines	5.26 minutes	25.9 seconds	6.05 seconds
99.9999%	six nines	31.5 seconds	2.59 seconds	0.605 seconds
99.99999%	seven nines	3.15 seconds	0.259 seconds	0.0605 seconds



SLA is a document

Technical Support: During the contract, we will provide support at no additional cost for the service(s) the customer has purchased. We provide customers with telephonic, email, and online chat support, noted below, excluding regional holidays (“**Business Hours**”).

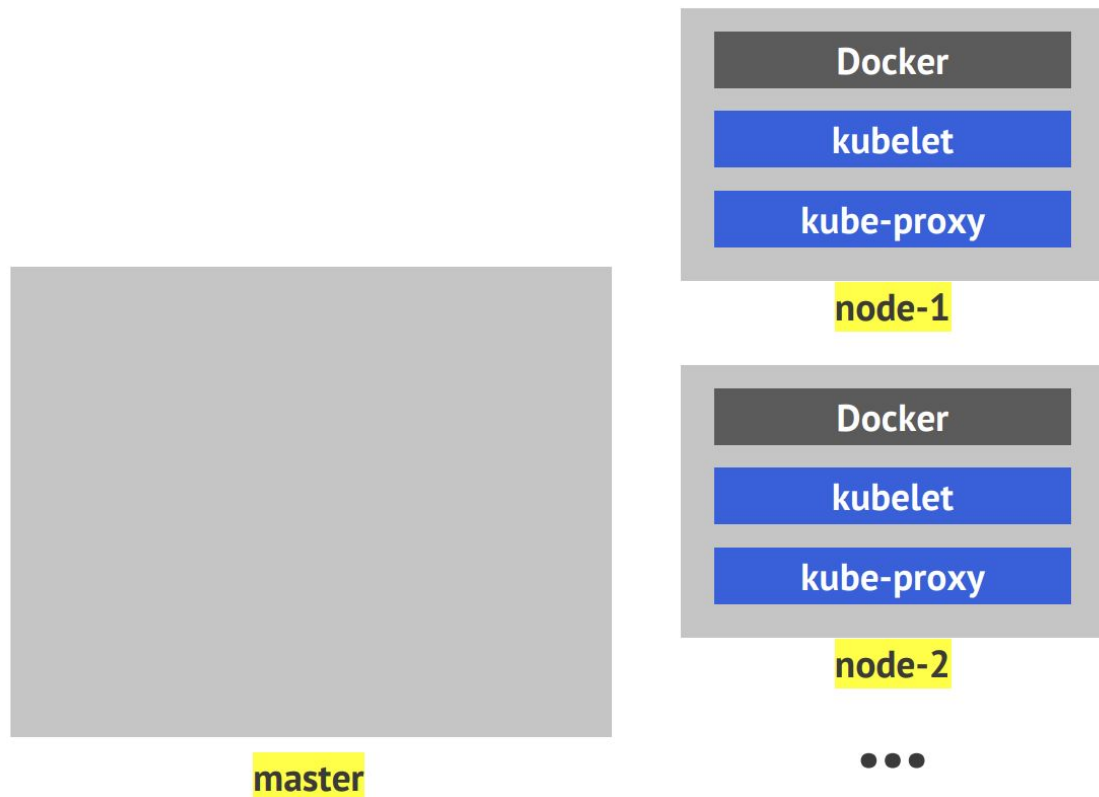
A support representative will determine the severity level based on the description provided by the customer contacts according to the table below. Customer contacts will ensure that each support case submitted will designate the initial severity level of the error in accordance with the definitions in the table, identify the services that experienced the error, and provide contact information for the customer contact most familiar with the issue.

SEVERITY LEVELS & TARGET RESPONSE TIMES

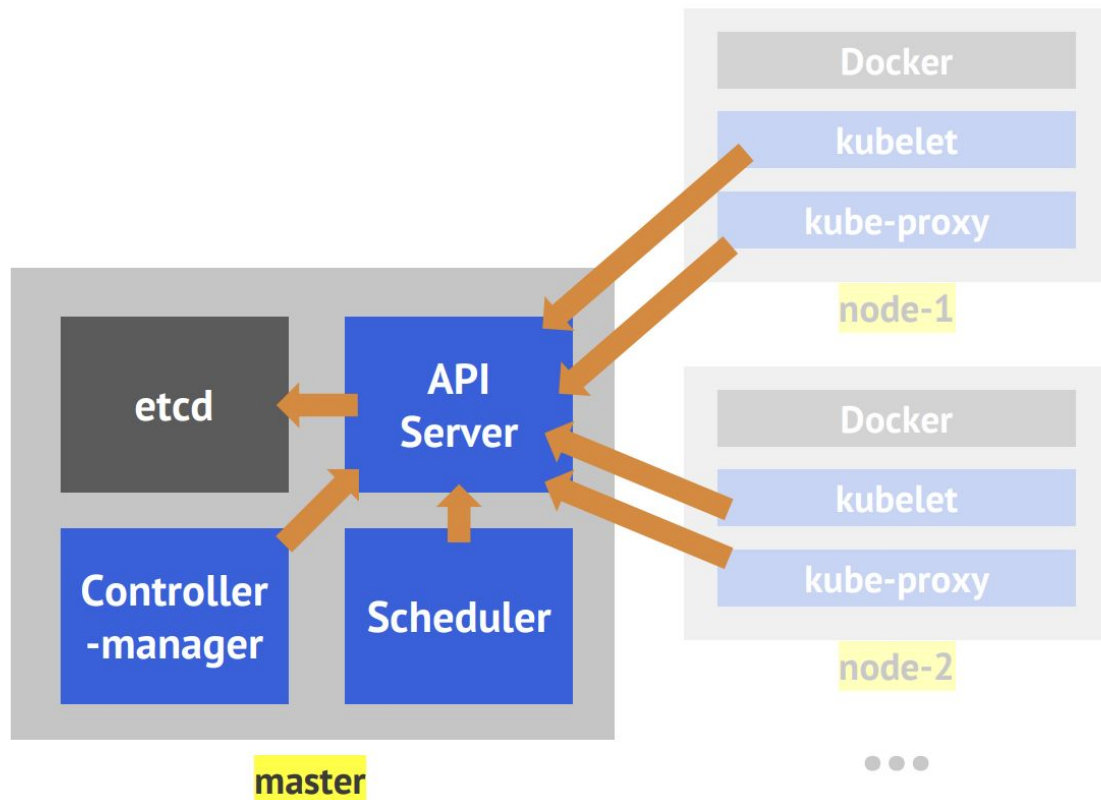
Priority Code	Priority Definition	Target Response Times
P1 – Urgent (Severity 1)	A critical failure in the operational activity of the Services, or an Error that causes the Services to be severely impacted or completely shut down, or Customer’s use of the Services is impossible, where no workaround is available.	2 hours
P2 – High (Severity 2)	Errors include high-impact issues in which the Services are inoperative or seriously degraded where a short-term workaround is available.	4 hours
P3 – Medium (Severity 3)	The Error limits the functionality or usefulness of the Services, but the condition is not critical to the continued operation of the Services. A workaround is readily available and can be applied or used with little or no operational impact.	6 hours
P4 – Low (Severity 4)	Minimal problems in the Services arising from a misleading or unsatisfactory component or feature. The problem can be circumvented with no operational impact and there are no data integrity issues.	8 Hours



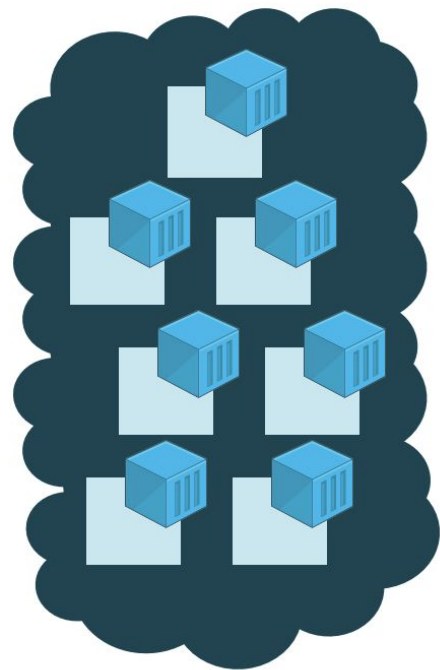
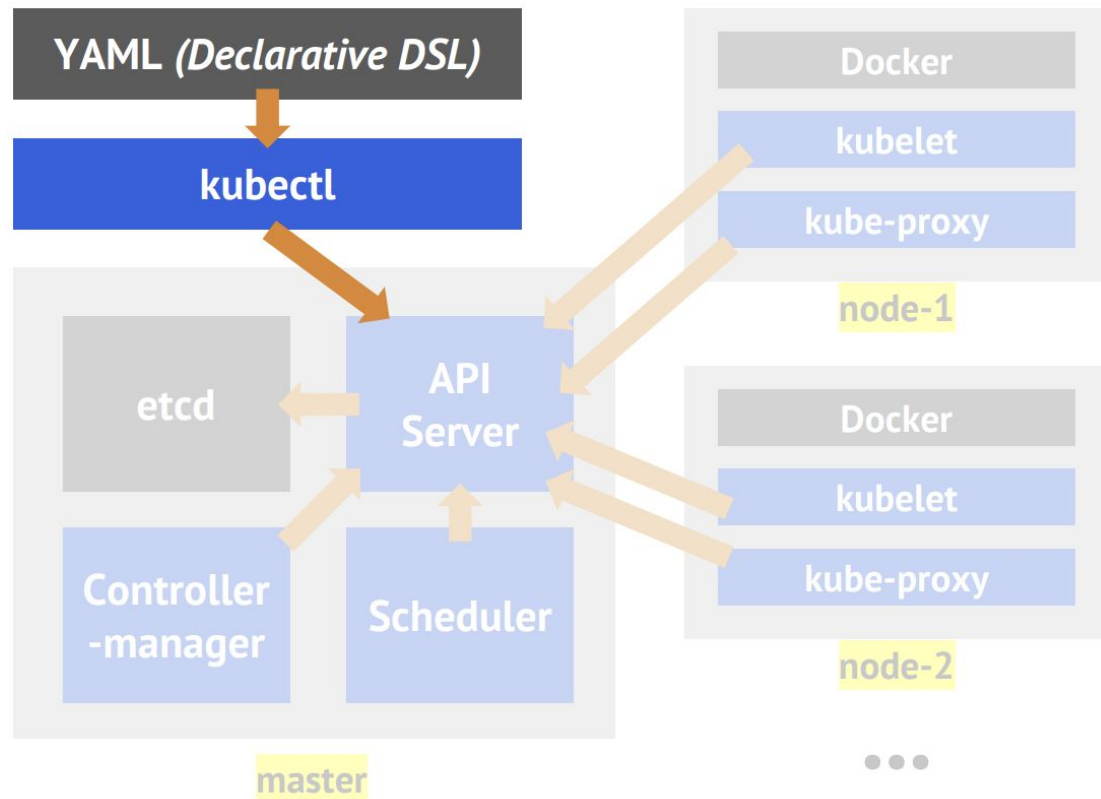
Архитектура Kubernetes



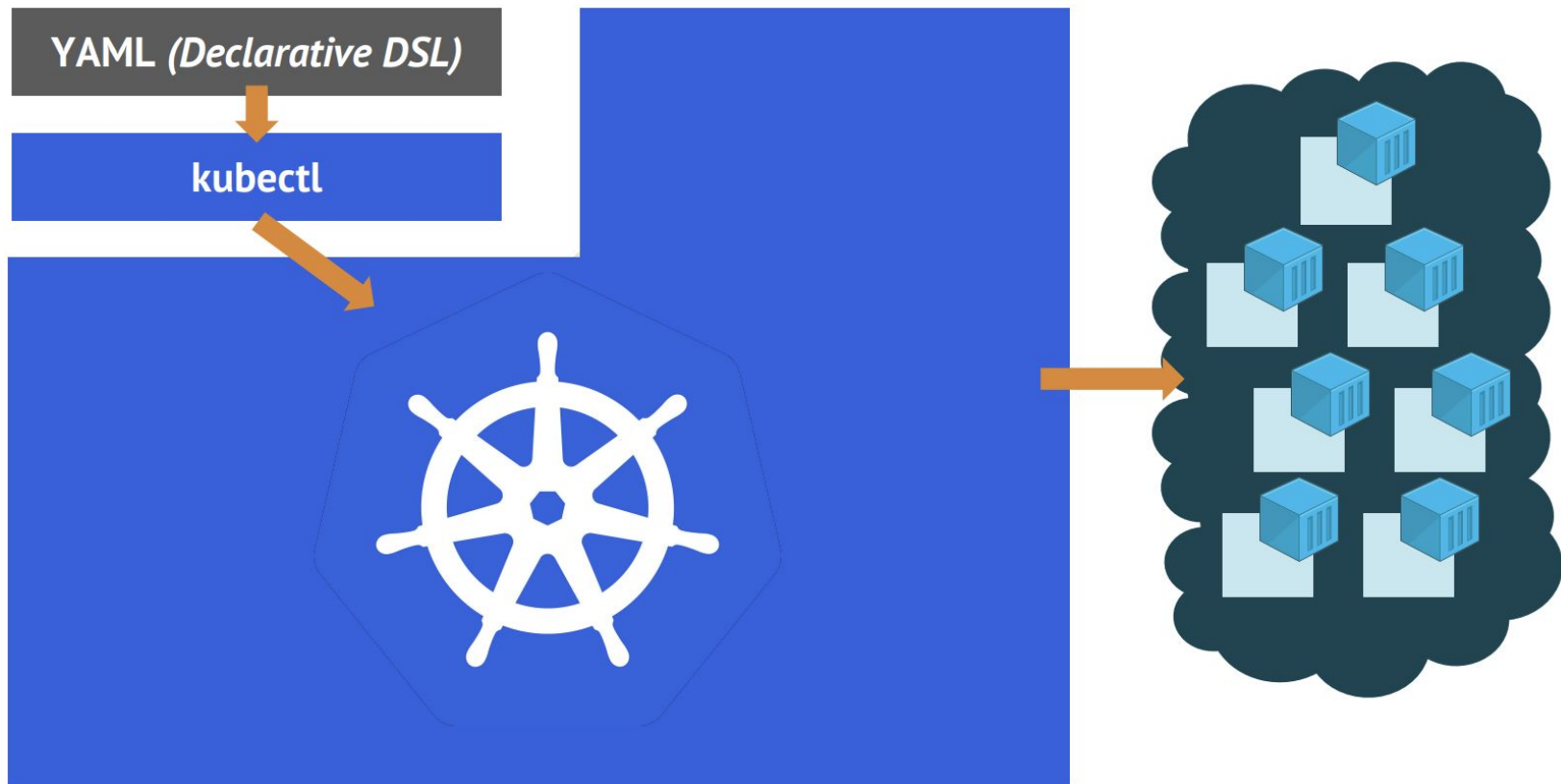
Архитектура Kubernetes



Архитектура Kubernetes



Архитектура Kubernetes

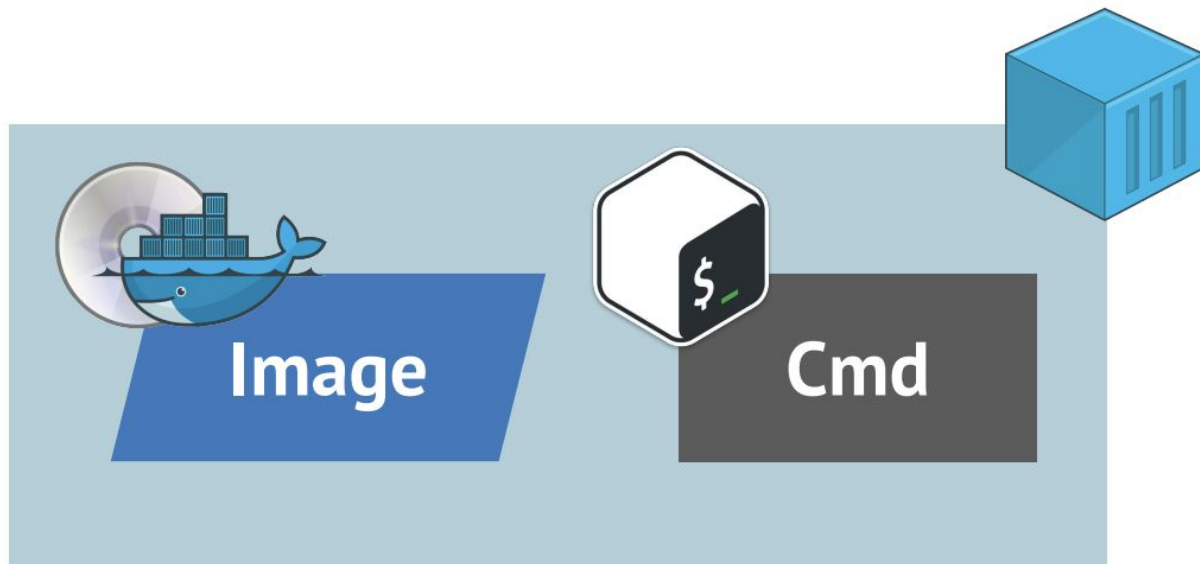


Container

Контейнер обеспечивает удобное:

- Логирование
- Перезапуски

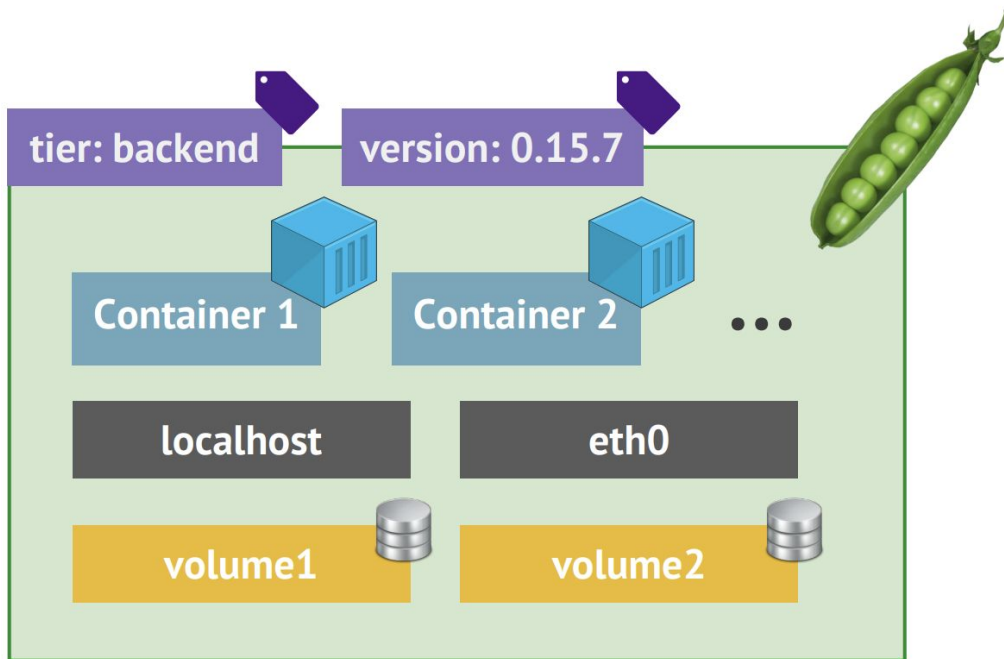
Нельзя запустить!



Pod

Несколько контейнеров с общей сетью и (возможно) папками. Минимальный примитив

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
spec:
  containers:
  - name: nginx
    image: nginx:1.14.2
    ports:
    - containerPort: 80
```



ReplicaSet

Несколько подов, k8s следит чтобы было запущено нужное количество

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: frontend
  labels:
    app: guestbook
    tier: frontend
spec:
  # modify replicas according to your case
  replicas: 3
  selector:
    matchLabels:
      tier: frontend
  template:
    metadata:
      labels:
        tier: frontend
    spec:
      containers:
        - name: php-redis
          image: us-docker.pkg.dev/google-samples/containers/gke/gb-frontend:v5
```

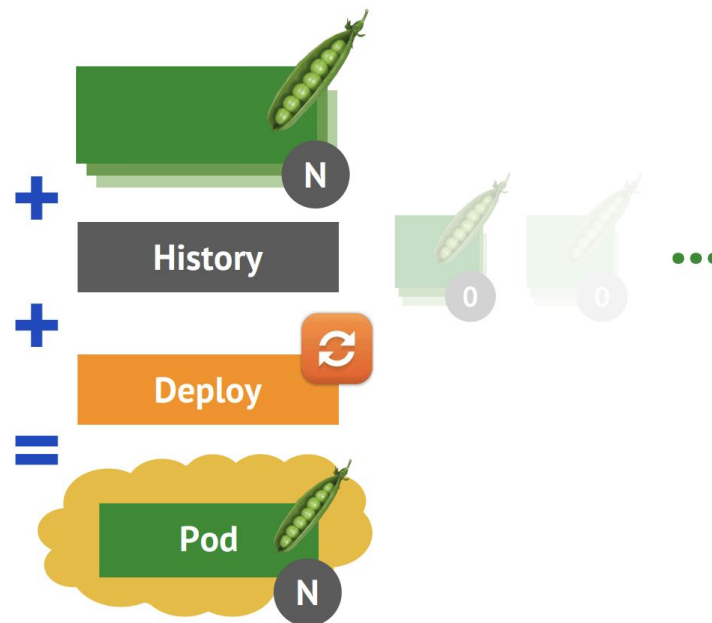


Deployment

ReplicaSet с возможностью RollingUpdate.

Все что до этого было использовать не нужно!

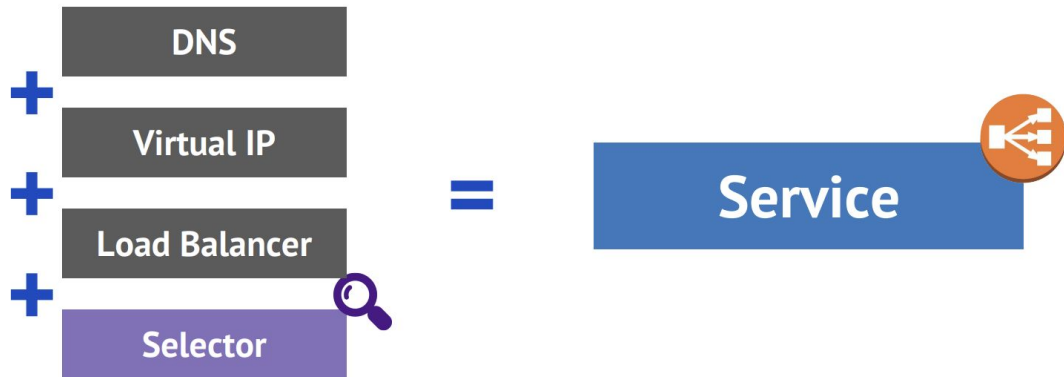
```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.14.2
          ports:
            - containerPort: 80
```



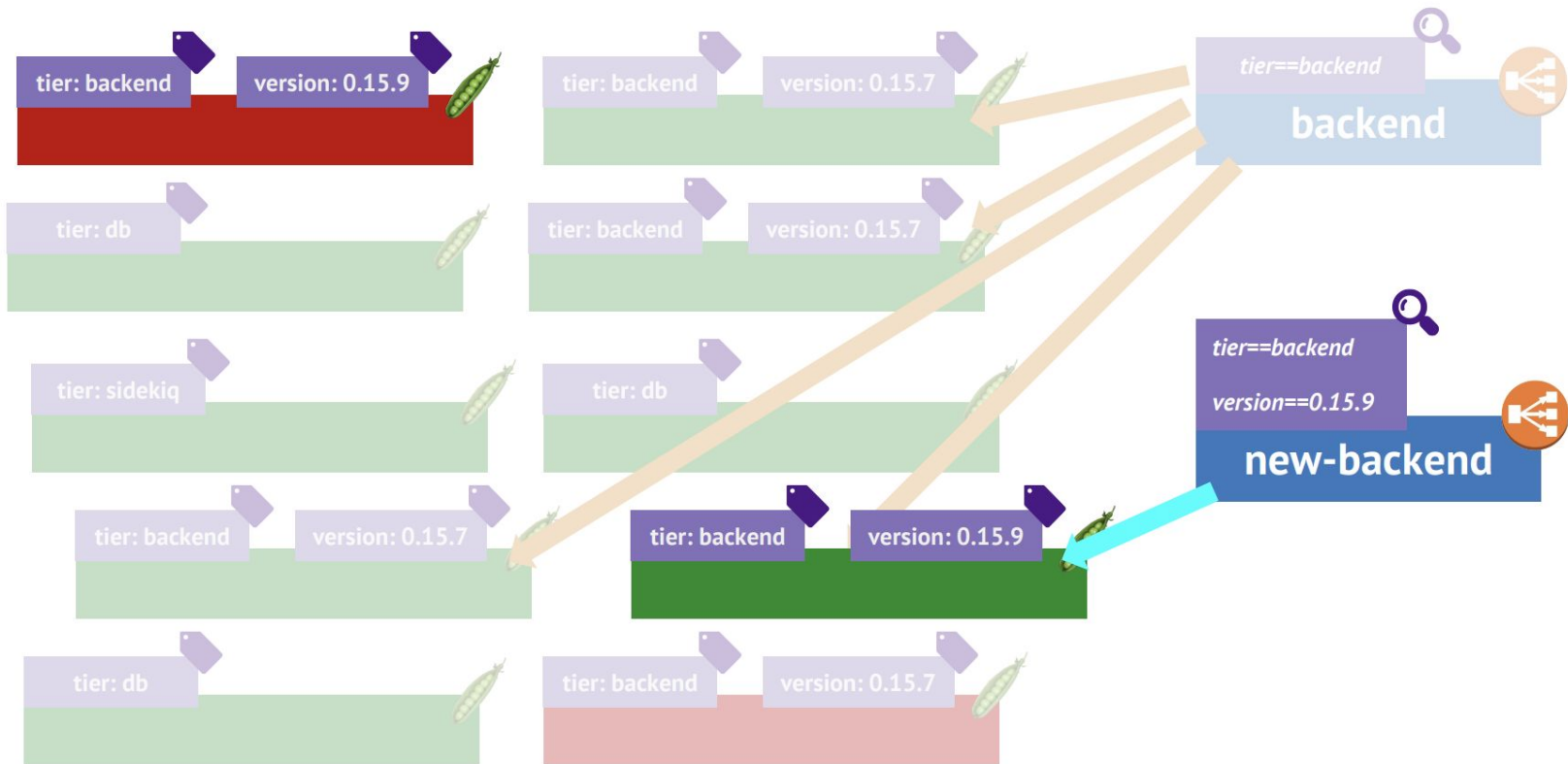
Service

Единый endpoint для группы подов, определяющихся селекторами

```
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  selector:
    app.kubernetes.io/name: MyApp
  ports:
    - protocol: TCP
      port: 80
      targetPort: 9376
```



Service



Job

Единоразово запускающийся под

```
apiVersion: batch/v1
kind: Job
metadata:
  name: pi
spec:
  template:
    spec:
      containers:
      - name: pi
        image: perl:5.34.0
        command: ["perl", "-Mbignum=bpi", "-wle", "print bpi(2000)"]
      restartPolicy: Never
  backoffLimit: 4
```



CronJob

Job по расписанию

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: hello
spec:
  schedule: "* * * * *"
  jobTemplate:
    spec:
      template:
        spec:
          containers:
            - name: hello
              image: busybox:1.28
              imagePullPolicy: IfNotPresent
              command:
                - /bin/sh
                - -c
                - date; echo Hello from the Kubernetes cluster
          restartPolicy: OnFailure
```

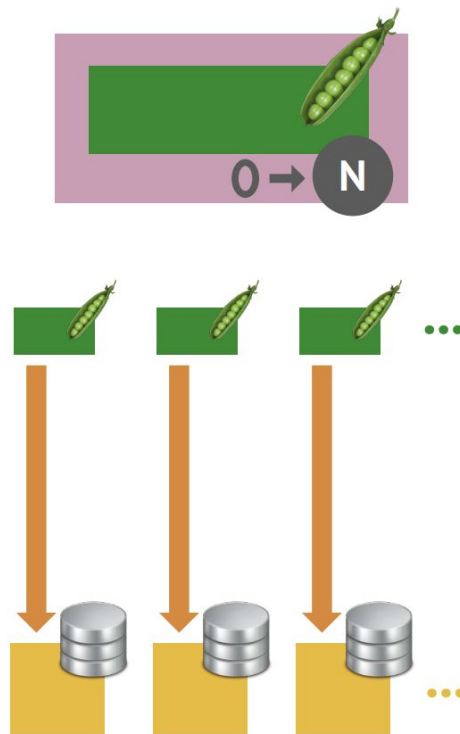


StatefulSet

Несколько подов, у каждого свое хранилище
(Persistent Volume)

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: web
spec:
  selector:
    matchLabels:
      app: nginx # has to match .spec.template.metadata.labels
  serviceName: "nginx"
  replicas: 3 # by default is 1
  minReadySeconds: 10 # by default is 0
  template:
    metadata:
      labels:
        app: nginx # has to match .spec.selector.matchLabels
    spec:
      terminationGracePeriodSeconds: 10
      containers:
        - name: nginx
          image: registry.k8s.io/nginx-slim:0.24
          ports:
            - containerPort: 80
            name: web
          volumeMounts:
            - name: www
              mountPath: /usr/share/nginx/html
      volumeClaimTemplates:
        - metadata:
            name: www
          spec:
            accessModes: [ "ReadWriteOnce" ]
            storageClassName: "my-storage-class"
            resources:
              requests:
                storage: 1Gi
```

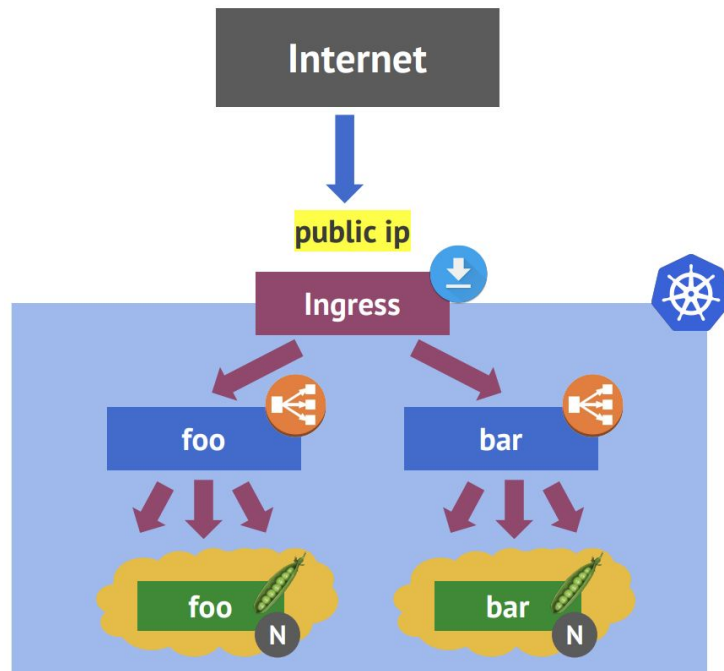
StatefulSet



Ingress

Единая точка входа в приложения, объединяет несколько сервисов

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: minimal-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx-example
  rules:
  - http:
      paths:
      - path: /testpath
        pathType: Prefix
        backend:
          service:
            name: test
            port:
              number: 80
```



ConfigMap/Secret

Хранение конфигов и секретов

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: game-demo
data:
  # property-like keys; each key maps to a simple value
  player_initial_lives: "3"
  ui_properties_file_name: "user-interface.properties"

  # file-like keys
  game.properties: |
    enemy.types=aliens,monsters
    player.maximum-lives=5
  user-interface.properties: |
    color.good=purple
    color.bad=yellow
    allow.textmode=true
```

```
apiVersion: v1
kind: Secret
metadata:
  name: dotfile-secret
data:
  .secret-file: dmFsdWUtMg0KDQo=
---
apiVersion: v1
kind: Pod
metadata:
  name: secret-dotfiles-pod
spec:
  volumes:
    - name: secret-volume
      secret:
        secretName: dotfile-secret
  containers:
    - name: dotfile-test-container
      image: registry.k8s.io/busybox
      command:
        - ls
        - "_l"
        - "/etc/secret-volume"
      volumeMounts:
        - name: secret-volume
          readOnly: true
          mountPath: "/etc/secret-volume"
```



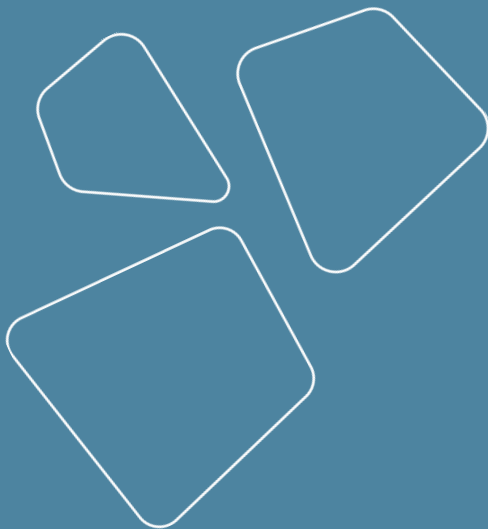
Kubernetes: Keywords

- [Документация](#)
- [minikube](#) - локально развернуть k8s
- [Namespace](#) - разделение проектов
- [NVIDIA GPU Operator](#) - управление GPU
- [Helm](#) - пакетный менеджер
- [Seldon Core](#), [KServe](#) - запускатели моделей
- [Docker Swarm](#) - для истории



Revise

1. Virtualization
2. Docker
 - a. Podman
 - b. Compose
3. Kubernetes



Thanks for attention!

Questions?

